

Möller–Trumbore intersection algorithm

The **Möller–Trumbore ray-triangle intersection algorithm**, named after its inventors Tomas Möller and Ben Trumbore, is a fast method for calculating the intersection of a ray and a triangle in three dimensions without needing precomputation of the plane equation of the plane containing the triangle.^[1] Among other uses, it can be used in computer graphics to implement ray tracing computations involving triangle meshes.^[2]

Calculation

Definitions

The ray is defined by an origin point O and a direction vector $\vec{v}$. Every point on the ray can be expressed by $\vec{r}(t) = O + t\vec{v}$, where the parameter t ranges from negative infinity to infinity. The triangle is defined by three vertices, named v_1 , v_2 , v_3 . The plane that the triangle is on, which is needed to calculate the ray-triangle intersection, is defined by a point on the plane, such as v_1 , and a vector that is orthogonal to every point on that plane, such as the cross product between the vector from v_1 to v_2 and the vector from v_1 to v_3 :

$\vec{n} \cdot (P_1 - P_2) = 0$, where $\vec{n} = (v_2 - v_1) \times (v_3 - v_1)$, and P_1 and P_2 are any points on the plane.

Check if the ray is parallel to the triangle

First, find out if the line produced by the ray intersects with the plane that the triangle is on, and if it does, find the coordinates of that intersection. The only way that the line will *not* intersect the plane is if the ray's direction vector is parallel to the plane.^[3] When this happens, the dot product between the ray's direction vector and the plane's normal vector will be zero. Otherwise, the line *does* intersect the plane *somewhere*, but not necessarily within the triangle.

Check if the ray-plane intersection lies outside the triangle

Using barycentric coordinates, any point on the triangle can be expressed as a convex combination of the triangle's vertices:

$$P = wv_1 + uv_2 + vv_3$$

The coefficients must be non-negative and sum to 1, so w can be replaced with $1 - u - v$:

$$\begin{aligned} P &= (1 - u - v)v_1 + uv_2 + vv_3 \\ P &= v_1 + u(v_2 - v_1) + v(v_3 - v_1) \end{aligned}$$

where P is any point on the plane. Observe that $\vec{e_1} = v_2 - v_1$ and $\vec{e_2} = v_3 - v_1$ are vectors on the edge of the triangle, and together, they span a plane (which goes through the origin). Each point on that plane can be written as $ue_1 + ve_2$ and can be translated by v_1 to "move" that point onto the plane that the triangle is on.

To find u and v for a particular intersection, set the ray expression equal to the plane expression, and put the variables on one side and the constants on the other.

$$\begin{aligned} O + tD &= v_1 + u(v_2 - v_1) + v(v_3 - v_1) \\ O - v_1 &= -tD + u(v_2 - v_1) + v(v_3 - v_1) \end{aligned}$$

This is a system of linear equations with three equations (one each for x, y, z) and three unknowns (t, u , and v), and can be represented as a matrix-vector multiplication.

$$\begin{bmatrix} | & | & | \\ -D & (v_2 - v_1) & (v_3 - v_1) \\ | & | & | \end{bmatrix} \begin{bmatrix} t \\ u \\ v \end{bmatrix} = O - v_1$$

This equation will always have a solution when the matrix has three linearly independent column vectors in $\mathbb{R}^3$ and is thus invertible. This happens if and only if the triangle vertices aren't collinear and the ray isn't parallel to the plane.

The algorithm can use Cramer's Rule to find the t, u , and v values for an intersection, and if it lies within the triangle, the exact coordinates of the intersection can be found by plugging in t to the ray's equation.

Implementations

C++ implementation

The following is an implementation of the algorithm in C++ using the glm library:

```
std::optional<vec3> ray_intersects_triangle( const vec3 &ray_origin,
const vec3 &ray_vector,
const triangle3& triangle)
{
    constexpr float epsilon = std::numeric_limits<float>::epsilon();

    vec3 edge1 = triangle.b - triangle.a;
    vec3 edge2 = triangle.c - triangle.a;

    // Backface culling, assuming CCW-wound triangles.
    const vec3 normal = cross(edge1, edge2); // No need to normalize
    if (dot(normal, ray_vector) > 0) return {};

    vec3 ray_cross_e2 = cross(ray_vector, edge2);
    float det = dot(edge1, ray_cross_e2);

    if (abs(det) < epsilon) return {}; // Ray is parallel to triangle

    float inv_det = 1.0 / det;
    vec3 s = ray_origin - triangle.a;
    float u = inv_det * dot(s, ray_cross_e2);

    if (u < -epsilon || u > 1 + epsilon) return {}; // Ray passes outside edge2's bounds

    vec3 s_cross_e1 = cross(s, edge1);
```

```

float v = inv_det * dot(ray_vector, s_cross_e1);

if (v < -epsilon || u + v - 1 > epsilon) return {}; // Ray passes outside edge1's bounds

// The ray line intersects with the triangle.
// We compute t to find where on the ray the intersection is.
float t = inv_det * dot(edge2, s_cross_e1);

if (t > epsilon) // Ray intersection
{
    return vec3(ray_origin + ray_vector * t);
}
else // This means that there is a line intersection but not a ray intersection.
    return {};
}

```

Rust implementation

The following is an implementation of the algorithm in Rust using the glam crate:

```

fn moller_trumbore_intersection (origin: Vec3, direction: Vec3, triangle: Triangle) -> Option<Vec3> {
    let e1 = triangle.b - triangle.a;
    let e2 = triangle.c - triangle.a;

    let ray_cross_e2 = direction.cross(e2);
    let det = e1.dot(ray_cross_e2);

    if det > -f32::EPSILON && det < f32::EPSILON {
        return None; // This ray is parallel to this triangle.
    }

    let inv_det = 1.0 / det;
    let s = origin - triangle.a;
    let u = inv_det * s.dot(ray_cross_e2);
    if u < 0.0 || u > 1.0 {
        return None;
    }

    let s_cross_e1 = s.cross(e1);
    let v = inv_det * direction.dot(s_cross_e1);
    if v < 0.0 || u + v > 1.0 {
        return None;
    }
    // At this stage we can compute t to find out where the intersection point is on the line.
    let t = inv_det * e2.dot(s_cross_e1);

    if t > f32::EPSILON { // ray intersection
        let intersection_point = origin + direction * t;
        return Some(intersection_point);
    }
    else { // This means that there is a line intersection but not a ray intersection.
        return None;
    }
}

```

Java implementation

The following is an implementation of the algorithm in Java using `javax.vecmath` from Java 3D API:

```

public class MollerTrumbore {

    private static final double EPSILON = 0.0000001;

    public static boolean rayIntersectsTriangle(Point3d rayOrigin,
                                                Vector3d rayVector,
                                                Triangle inTriangle,
                                                Point3d outIntersectionPoint) {
        Point3d vertex0 = inTriangle.getVertex0();
    }
}

```

```

Point3d vertex1 = inTriangle.getVertex1();
Point3d vertex2 = inTriangle.getVertex2();
Vector3d edge1 = new Vector3d();
Vector3d edge2 = new Vector3d();
Vector3d h = new Vector3d();
Vector3d s = new Vector3d();
Vector3d q = new Vector3d();
double a, f, u, v;
edge1.sub(vertex1, vertex0);
edge2.sub(vertex2, vertex0);
h.cross(rayVector, edge2);
a = edge1.dot(h);

if (a > -EPSILON && a < EPSILON) {
    return false;    // This ray is parallel to this triangle.
}

f = 1.0 / a;
s.sub(rayOrigin, vertex0);
u = f * (s.dot(h));

if (u < 0.0 || u > 1.0) {
    return false;
}

q.cross(s, edge1);
v = f * rayVector.dot(q);

if (v < 0.0 || u + v > 1.0) {
    return false;
}

// At this stage we can compute t to find out where the intersection point is on the line.
double t = f * edge2.dot(q);
if (t > EPSILON) // ray intersection
{
    outIntersectionPoint.set(0.0, 0.0, 0.0);
    outIntersectionPoint.scaleAdd(t, rayVector, rayOrigin);
    return true;
} else // This means that there is a line intersection but not a ray intersection.
{
    return false;
}
}
}

```

See also

- [Badouel intersection algorithm](#)

References

1. Möller, Tomas; Trumbore, Ben (1997). "Fast, Minimum Storage Ray-Triangle Intersection". *Journal of Graphics Tools*. **2**: 21–28. doi:10.1080/10867651.1997.10487468 (https://doi.org/10.1080%2F10867651.1997.10487468).
2. "Ray-Triangle Intersection" (<http://www.lighthouse3d.com/tutorials/maths/ray-triangle-intersection>). lighthouse3d. 26 March 2011. Retrieved 2017-09-10.
3. Note: If the ray's origin is itself on the plane, in addition to the ray's direction vector being parallel to the plane, than the entire ray is technically on the plane. However, since theoretical planes are infinitely thin, the ray would still be considered to not intersect the plane in that scenario.

External links

- Fast Minimum Storage Ray-Triangle Intersection (<https://www.graphics.cornell.edu/pubs/1997/MT97.pdf>)
 - Optimizations on the basic algorithm by Möller & Trumbore (https://github.com/erich666/jgt-code/tree/master/Volume_02/Number_1/Moller1997a), code from *journal of graphics tools*
 - Ray-Tracing: Rendering a Triangle (<https://www.scratchapixel.com/lessons/3d-basic-rendering/ray-tracing-rendering-a-triangle/moller-trumbore-ray-triangle-intersection.html>)
 - MATLAB version (<http://www.mathworks.com/matlabcentral/fileexchange/33073>) of this algorithm (highly vectorized)
 - Baldwin-Weber ray-triangle intersection algorithm (<http://jcgt.org/published/0005/03/03/>)
 - Schlick–Subrenat algorithm^[1] for ray-quadrilateral intersection
1. *Ray Intersection of Tessellated Surfaces: Quadrangles versus Triangles* (<http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.44.6955>), Schlick C., Subrenat G. Graphics Gems 1993
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=Möller–Trumbore_intersection_algorithm&oldid=1347436352"